



PDF Download
3291280.3291800.pdf
12 January 2026
Total Citations: 22
Total Downloads: 847

 Latest updates: <https://dl.acm.org/doi/10.1145/3291280.3291800>

RESEARCH-ARTICLE

A Deep Look into Logarithmic Quantization of Model Parameters in Neural Networks

JINGYONG CAI, Tokyo University of Agriculture and Technology, Fuchu, Tokyo, Japan

MASASHI TAKEMOTO

HIRONORI NAKAJO, Tokyo University of Agriculture and Technology, Fuchu, Tokyo, Japan

Open Access Support provided by:

Tokyo University of Agriculture and Technology

Published: 10 December 2018

[Citation in BibTeX format](#)

IAIT 2018: The 10th International
Conference on Advances in Information
Technology
December 10 - 13, 2018
Bangkok, Thailand

A Deep Look into Logarithmic Quantization of Model Parameters in Neural Networks

Jingyong Cai
Graduate School of Engineering,
Tokyo University of Agriculture and
Technology
Tokyo, Japan
s174740v@st.go.tuat.ac.jp

Masashi Takemoto
BeatCraft Inc
Tokyo, Japan
lesser@beatcraft.com

Hironori Nakajo
Institute of Engineering, Tokyo
University of Agriculture and
Technology
Tokyo, Japan
nakajo@cc.tuat.ac.jp

ABSTRACT

Based on the fact that parameters of pre-trained neural networks naturally have non-uniform distributions, logarithmic quantization of network parameters achieves better classification results than linear quantization of the same resolution. In our practice, we found that the logarithmic quantization suffers huge accuracy decrease on small size neural networks. This is because the parameters of trained small neural networks are not highly concentrated around 0. In this paper, we analyse in depth the attributes of logarithmic quantization. In addition, existing compression algorithms highly rely on retraining which requires heavy computational power. In such a situation, we propose a new logarithmic quantization algorithm to mitigate the deterioration on neural networks which contain layers of small size. As the result, our method achieves the minimum accuracy loss on GoogLeNet after direct quantization compared to quantized counterparts.

CCS CONCEPTS

• Computing methodologies → Neural networks;

KEYWORDS

Neural Networks Compression, Logarithmic Quantization, Computer Vision, Deep Learning.

ACM Reference Format:

Jingyong Cai, Masashi Takemoto, and Hironori Nakajo. 2018. A Deep Look into Logarithmic Quantization of Model Parameters in Neural Networks. In *The 10th International Conference on Advances in Information Technology (IAIT 2018)* (IAIT 2018), December 10–13, 2018, Bangkok, Thailand. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3291280.3291800>

1 INTRODUCTION

1.1 Background

Deep neural networks show their incomparable performance in computer vision, natural language processing, visual art processing and so forth. However, on one hand, since training of deep

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
IAIT 2018, December 10–13, 2018, Bangkok, Thailand
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-6568-0/18/12...\$15.00
<https://doi.org/10.1145/3291280.3291800>

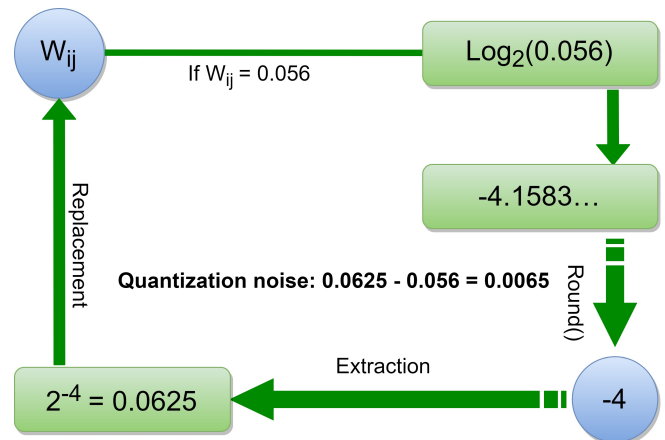


Figure 1: An overview of logarithmic quantization.

neural networks is extremely time and resource consuming, many researchers are committed to reducing the amount of computation so as to train neural networks fast and easily [18][21][22]. On the other hand, the huge storage cost of trained neural networks hinders their deployments on mobile or IoT devices. The era of 5G is approaching, and it is foreseeable that IoT devices will be more commonly used in all walks of life. Nonetheless, deep neural networks are typically hard to deployed on an IoT device due to its small RAM size [2]. To deal with it, various quantization and pruning algorithms were designed to reduce the memory requirements. Among all quantization approaches, we choose to delve into logarithmic quantization since the similar non-uniformity between the weight value and log-2 based weight encoding brings better accuracy. Moreover, existing pruning and quantization algorithms highly rely on retraining which requires heavy computational power. To address it, we propose a new retraining-free logarithmic quantization algorithm. Our algorithm was tested on small complexity networks and still outputs superior performance without retraining.

1.2 Overview of Logarithmic Quantization

The main purpose of quantization is compressing the original deep neural networks. Quantization is an effective way to reduce memory consumption and has recently been adopted by Google's TensorFlow [1]. Various compression methods will be detailed in section 2, and here we give a brief overview of logarithmic quantization (figure 2) proposed by Miyashita et al. [20]. Note the rounding and clipping schemes will be detailed in section 3. As we can see

from figure 1, logarithmic quantization algorithm takes logarithms of the original weights and saves only exponents. Besides, to constrain the exponents within a smaller bitwidth, the decimal parts of the exponents are removed. As a trade-off, quantized weights incur relatively large quantization noises. To decrease the accuracy deterioration of quantized neural networks, despite troublesome, retraining of quantized networks is widely used and often plays an important role.

1.3 Visualization of Quantized Weights

Given different bitwidths, the dynamic range of quantized weights varies. Supposing the given bitwidth is 4 then we have at most 16 exponents. Thus we get quantized weights as the equation (1). Note ‘s’ in the equation refers to a global parameter used to adjust the dynamic range of the quantized weights. This power of 2 based encoding also enables Zhou et al.’s INQ (Incremental Network Quantization) [27] to function.

$$\text{QuanWeights} = \{\pm 2^{0+s}, \pm 2^{-1+s} \dots \pm 2^{-16+s}\} \quad (1)$$

To compare the weights distribution before and after logarithmic quantization, we trained a fully connected neural network on MNIST [5]. The network contains 2 hidden layers with 1000 neurons each. We set s to 0 and the weights distribution is illustrated in figure 2 before and after quantization on the first hidden layer. It is clear that quantized weights on two sides are sparse. This phenomenon is attributed to the adoption of integer-only exponents. More specifically, only 3 quantized weights between 0.1 and 1 namely 2^{-1} , 2^{-2} and 2^{-3} . Although the accuracy decrease roots from the sparsity on bigger weights can be alleviated through retraining, it does not match our goal of retraining-free.

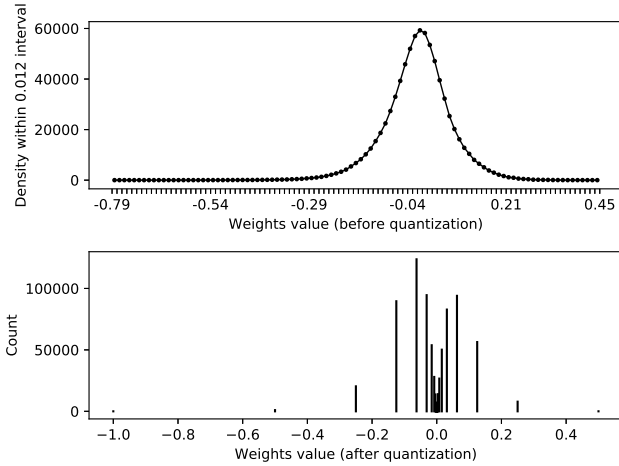


Figure 2: Distribution of weights before and after logarithmic quantization.

2 RELATED WORKS

2.1 Pruning

Pruning is an effective way to reduce network size. Removing some unimportant connections from a pre-trained neural network

leads to lossless accuracy. More interestingly, network pruning has biological foundations. Back in 1997, Chechik et al.’s work revealed that brain development involves selective pruning [3]. Lately Han et al. apply pruning to artificial neural networks without affecting the accuracy [7][8]. In their works, weights below a certain threshold are removed (the specific threshold is unknown).

How to evaluate the importance of each neuron or connection is the core of pruning. The easiest way to perform pruning is removing weights of small magnitude. As for why it works, it is primarily because small weights contribute less during forward computation. Other pruning strategies also have been extensively researched. For instance, ThiNet [19] performs pruning based on the statistical information computed from the next layer. Even pruning before the training step can offer effective learning [16].

2.2 Compression and Retraining

To compress neural networks, Courbariaux et al. [10] introduced BNN (binary neural network) which constrains all weights into +1 and -1. By retraining, BNN achieves nearly state-of-the-art accuracy on CIFAR-10, MNIST and SVHN though deteriorates drastically on ImageNet [4] dataset (22.1% of Top-5 accuracy decrease on GoogLeNet) [11]. Either Binary or Ternary (+1, 0, -1) neural networks, at cost of huge quantization noises, rely deeply on retraining [24]. Hubara et al. also extended their experiments using Miyashita et al.’s logarithmic quantization algorithm which narrowed the accuracy gap to 8.1%. In another way, Deep Compression [7] clusters weights via K-means. Weights grouped into the same category will share a same quantized weight. Specifically, Deep Compression quantizes convolution layers’ weights to 8 bits and fully connected layers’ weights to 5 bits. After retraining they realized lossless accuracy. Combined with pruning and Huffman coding, Han et al. further reduced parameter size. Compared with Deep Compression which performs pruning and quantization in separate steps, CLIP-Q [26] carries out these 2 approaches in parallel during training. Recently Google’s team implements integer arithmetic only inference (see [13]) which quantize all weights into 8 bits integers. Integer only inference is realized via affine mapping, and now it is the quantization scheme adopted by TensorFlow Lite [6].

Modifying the existing architecture to reduce parameter size is also studied. Iandola et al. created SqueezeNet [12] which achieves AlexNet-level accuracy with 50× fewer parameters. SqueezeNet decreases the number of input channels to 3×3 filters. Besides, it also replaces 3×3 filters with 1×1 filters. SimpleNet [9] is another case, Hasanpour et al. released their 13 layers convolutional network in 2016 outperforming deeper and heavier networks. Interestingly SimpleNet avoids using 1×1 filters in early layers to preserve locality information.

Either pruning or quantization might cause a huge influence on the final accuracy. To restore the original accuracy, retraining is usually performed after network compression. Some quantization or pruning strategies are applied during the training process (see [19][28][27]) therefore no retraining is needed. The convergence of training using coarsely quantized weights was analysed by Li et al. [17]. On the other hand, direct quantization on pre-trained neural networks relies deeply on retraining. Typically, retraining algorithms still keep a copy of full-precision weights. Sung et al.’s

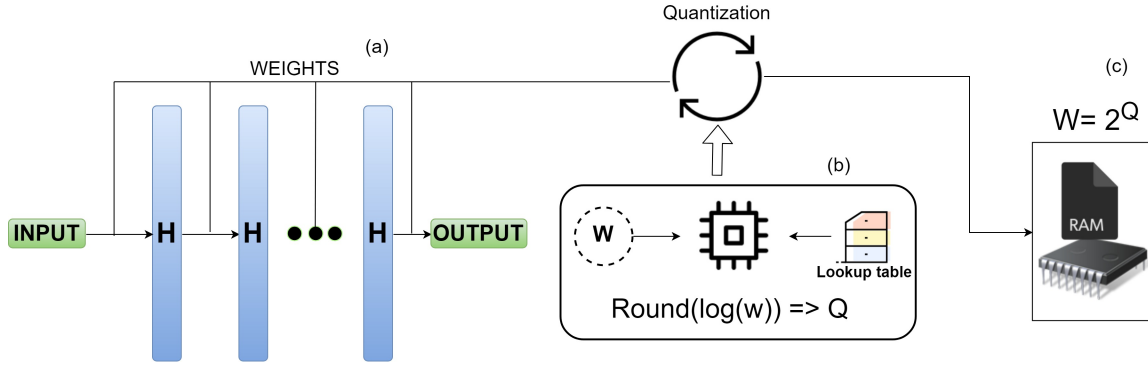


Figure 3: Processes of logarithmic quantization. (a) In the first step, weights are extracted for quantization. (b) Weights are quantized. Quantization algorithm takes logarithm of each weight and then round it to an integer. (c) Quantized weights are loaded to the target device. Since quantized weights consume less storage, inference on small ram devices could be accelerated.

retraining method [24] applies weight update only to full-precision weights and then generates quantized weights. They also assert that this approach takes less time to converge. Other efforts such as INQ [27] quantizes networks incrementally. In each iteration it quantizes only part of the network and adapt update to the left part. Although retraining algorithms can restore the original accuracy, they usually require big computational power since back propagation is involved.

3 LOGARITHMIC QUANTIZATION

3.1 Algorithm 1

Weights of trained neural networks with large complexity are usually highly concentrated around zero. This non-uniformity motivates Miyashita et al. to replace original parameters using logarithmic data representation since logarithmic quantized weights have similar trend. We restate Miyashita et al.’s algorithm as the following and a graphical view of logarithmic quantization processes is given in figure 3. Their algorithm (details in [20]) will be used in section 4 for performance comparison.

$$\text{LogQuant}(x, \text{bitwidth}, \text{FSR}) = \begin{cases} 0 & x=0, \\ 2^{\tilde{x}} & \text{otherwise} \end{cases} \quad (2)$$

where

FSR: Full scale range,

$$\tilde{x} = \text{Clip}(\text{Round}(\log_2 |x|), \text{FSR} - 2^{\text{bitwidth}}, \text{FSR}) \quad (3)$$

$$\text{Clip}(x, \text{min}, \text{max}) = \begin{cases} 0 & x \leq \text{min}, \\ \text{max}-1 & x \geq \text{max}, \\ x & \text{otherwise} \end{cases} \quad (4)$$

In addition, we define the rounding to the nearest scheme as the following:

$$\text{Round}(x) = \begin{cases} \lfloor x \rfloor & \text{if } (2^x - 2^{\lfloor x \rfloor}) \leq \frac{2^{\lfloor x \rfloor} - 2^{\lfloor x \rfloor - 1}}{2} \\ \lceil x \rceil & \text{if } (2^x - 2^{\lfloor x \rfloor}) > \frac{2^{\lfloor x \rfloor} - 2^{\lfloor x \rfloor - 1}}{2} \end{cases} \quad (5)$$

Quantized weights are given by Clip (equation (3)) and the FSR parameter is tuned during quantization process. The algorithm quantizes both activations and weights in order to replace multiplication operations with accumulations. In our practices, we found some drawbacks of algorithm 1 which are summarized as the following:

- 4bits or larger bit-width quantization brings little to no accuracy improvement than 3 bits quantization.
- Manually fine tuning the FSR (full scale range) parameter is cumbersome.
- Using only integers in log domain suffers accuracy decrease under some conditions since it penalizes strong connections (connections with greater weight value).
- Quantization of activations would incur local quantization processing on target devices.

3.2 Algorithm 2

The weak points mentioned in the last section might restrict the real world implementations of the proposed logarithmic quantization algorithm. Our purpose is to develop a new algorithm that keeps the advantages of the existing logarithmic quantization, while this algorithm shall get rid of the disadvantages from algorithm 1. To achieve this goal, we extend the exponents from integer domain to the real number field thus quantized weights now have a higher density. Extended exponents which beyond the precision of given bitwidth are kept in a lookup table. Moreover, We remove quantization of activations since quantized weights can be directly loaded into small devices of low computational power, while quantization of activations will incur local computing for quantization on these devices. Another benefit we gain from removing quantization of activations is that the time of tuning FSR parameter is largely saved. Therefore the generality of the algorithm is significantly increased.

In addition, we round all weights below a threshold to a minimum weight value. Stronger connections often perform prior roles in the forward computation. To better illustrate this point, we did some pruning tests on several networks. Making the use of Chainer [25] we take a snapshot of all parameters in a 2 layers FC (fully connected) network trained on MNIST, VGG [23] on Cifar10 and

GoogLeNet on ImageNet dataset. Note networks we used are pre-trained ones from caffe’s model zoo [14]. We pruned certain proportions of stronger connections in a certain layer and record the result. Then we repeated the same experiment but this time we pruned the weaker connections of the same proportion. Specifically, the FC network contains 2 hidden layers of 1000 neurons in each and we pruned 10 percent connections in the second hidden layer. On the GoogLeNet, 10 percent connections in the second convolution layer was pruned and to our surprise, loss of 10 percent stronger connections would dramatically decrease the accuracy. Another interesting fact is that the accuracy loss on pruned VGG was inconspicuous until we pruned 30 percent of the connections. It is evident that VGG on Cifar10 is over-parameterized. The accuracy loss is presented in table 1.

Table 1: Accuracy Loss After Pruning

Model	Upper pruning	Lower pruning
FC Network	38.56%	0%
GoogLeNet	78.45%	0%
VGG	11.19%	0%

The unimportance of smaller weights inspired us to round weights below a threshold to a specific minimum value. Here we provide details of our algorithm. We firstly generate an arithmetic progression via equation (6). Parameter R is a positive real number. In our cases, we set R to 7 on account of the fact that quantized values range from 0 to -7 can be decoded to weights range from $7.8e-3$ to 1. In practice, this range is appropriate for our tested networks and R could be tuned based on the complexity of a specific layer.

$$\text{QuanSeries} = \left\{ \frac{-Rx}{2^n} \mid (x \in 0 \sim 2^n) \cap (x \in \mathbb{Z}), n=\text{bitwidth} \right\} \quad (6)$$

Secondly, produced series contains float numbers which cannot be stored within the given bitwidth. Hence a lookup table is utilized to store these exponents. In this way, higher bitwidth offers denser exponents which is particularly useful for strong connections. Note that when the bitwidth is 3 our algorithm performs similar to algorithm 1. For the simplicity of implementation, we do not prune all connections below the threshold but round them to the bottom of the arithmetic progression, namely 2^{-7} . We will show in section 4 that this rounding scheme dose not affect the accuracy.

After quantization, only indexes of the series are stored in the memory. We use Numpy’s argmin[15] method to perform rounding to the nearest scheme (Listing 1). Note that the ‘-’ operator would be executed element-wisely. We ignore the process of fetching weight signs for the ease of explanation. The full quantization kernel could be found in the appendix.

Listing 1: Generating Indexes

```
Indexes = (QuanSeries - numpy.log2(numpy.abs(W))).argmin()
// W: weight matrices
```

Indexes could be directly loaded to target devices. Before inference on targets devices, the quantized weights should be reconstructed via equation (7). ‘ $\tilde{W}$ ’ denotes the quantized weights.

$$\tilde{W} = 2^{\text{QuanSeries}[\text{Indexes}]} \quad (7)$$

An overview of our algorithm is given in figure 4. Compared with algorithm 1 (figure 1), algorithm 2 can round original weights to a quantized version with smaller quantization noise. Figure 5, 6 and 7 show weight distributions after 4 or 5 bits quantization. Weights are extracted from the same layer of the pre-trained FC network (shown in figure 2). Compared with figure 2, it is notable that our algorithm outputs a more reasonable distribution especially for stronger connections.

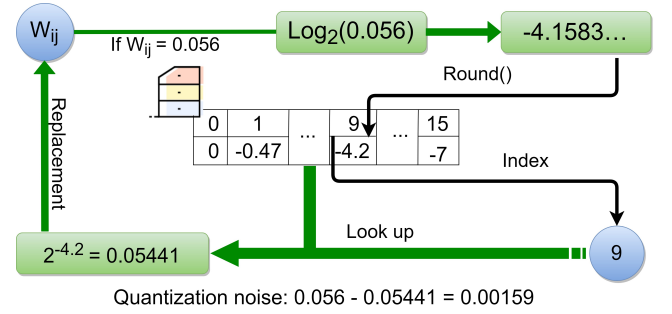


Figure 4: An overview of algorithm 2.

It is clear that increasing the bitwidth can solve the sparsity of strong connections in our algorithm. Particularly, figure 2 and figure 6 show almost no difference. On the contrary, from figure 5 and 7 we know algorithm 2 can increase the density of quantized weights. The ability to maintain the coherence of strong connections between original and quantized weights is the key to realize retraining-free quantization. Owing to its simplicity, we realize the quantization kernel within 20 lines of code (see listing 2 in the appendix).

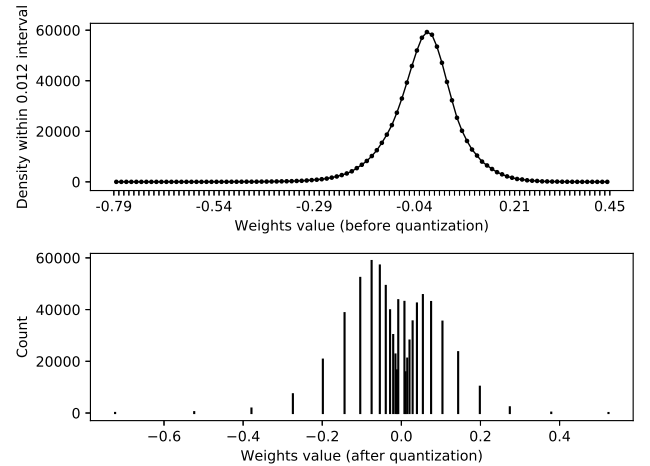


Figure 5: 4 bits quantization via algorithm 2.

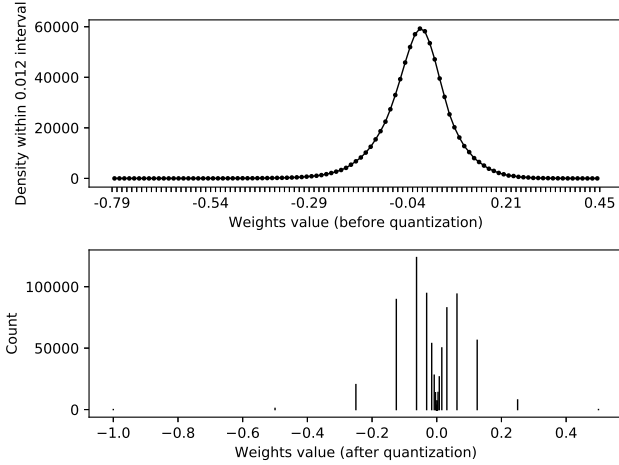


Figure 6: 5 bits quantization via algorithm 1. The distribution of quantized weights shows almost no difference with figure 2.

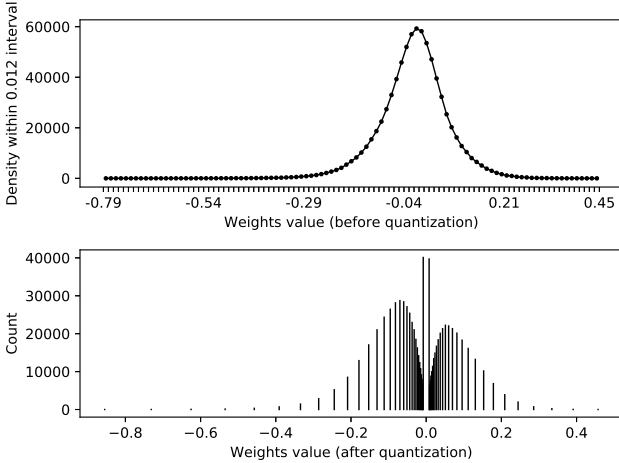


Figure 7: 5 bits quantization via algorithm 2.

3.3 Quantization Error

Quantization error (equation (8)) is used to measure the performance of quantization algorithm in [20] and [24], whereas we would argue that it is inaccurate to use equation (8) as the evaluation criterion especially for over-parameterized networks. Equation (8) treats all weights equally while we already know that pruning different parts of a neural network will cause different effects. When all connections are treated equally, the prior role of stronger connections is ignored. Nonetheless, we measured the mean squared error of two algorithms on a pre-trained VGG network and the results are collected in table 2. Obviously algorithm 1 outperforms algorithm 2 if we judge through their quantization error. However, as we will demonstrate in section 4, our algorithm still surpasses

algorithm 1. We can then conclude that over-parameterized networks are more capable of noise tolerance. Though quantization error of certain layers is heavier, it wouldn't impose a huge impact on final accuracy.

$$E = \frac{1}{2} \sum_{i=1}^N (Q(w_i) - w_i)^2 \quad (8)$$

Table 2: Quantization Noise (4 Bits Quantization)

VGG	Weights Number (K)	Alg1	Alg2
conv1_1	1.7	0.300	0.079
conv1_2	36.9	0.501	0.228
conv2_1	73.7	0.802	0.384
conv2_2	147.5	1.274	0.710
conv3_1	294.9	1.664	1.455
conv3_2	589.8	1.893	3.206
conv3_3	589.8	1.870	3.311
conv4_1	1179.6	1.840	8.816
conv4_2	2359.3	1.064	28.395
conv4_3	2359.3	0.369	44.201
conv5_1	2359.3	0.139	53.469
conv5_2	2359.3	0.148	50.590
conv5_3	2359.3	0.201	51.053
fc1	26.2	0.154	4.057
fc2	5.1	0.599	0.214

4 BENCHMARK RESULTS

At first, a simple FC network with 2 hidden layers is used to explore the limits of our algorithm. We gradually decrease the number of hidden layer neurons thus the size of each layer decreases respectively. Then the network is quantized using 2 algorithms. The results of 2 algorithms are gathered in table 3.

Table 3: 2 Layers fully connected network on MNIST

Hidden Layer Size	Alg1 Accuracy	Alg2 Accuracy	Ref
1000	0.9831	0.9846	0.9841
600	0.9796	0.9801	0.9792
200	0.9798	0.9814	0.9811
100	0.9726	0.9753	0.9751
80	0.9748	0.9744	0.9751
60	0.9611	0.9683	0.9723
40	0.9510	0.9690	0.9701
20	0.9159	0.9550	0.9626
10	0.8056	0.9300	0.9376

It is noticeable that even when hidden layers contain only 10 neurons, algorithm 2 can still keep close to the original accuracy. We also benchmarked our algorithm on VGG and GoogLeNet. Note the size of the first and the last layer in normal convolution networks is relatively small. Since layers of small complexity are weak in

noise tolerance, Miyashita et al. and Zhou et al. [29] chose not to quantize the first layer. Another reason is that the first layer often contain only a few channels and constitutes a small proportion of total computation [29]. Thus it does not affect much without quantizing the first layer. As shown in figure 8, compared with the second convolution layer, weights in layer 1 are not as concentrated as layer 2 and also feature a higher dynamic range.

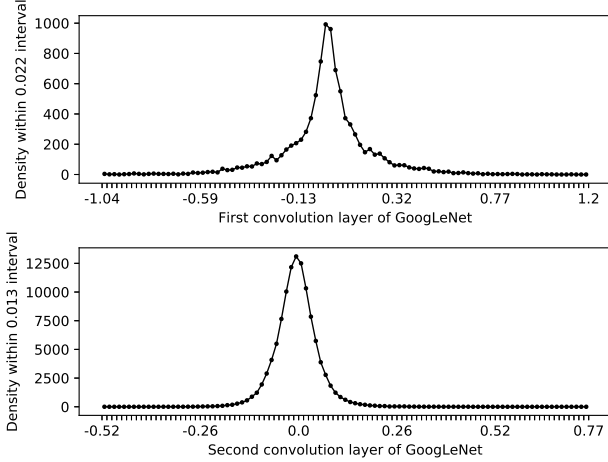


Figure 8: Weights distribution of the first two layers in GoogLeNet.

Rather than skipping the first layer, we quantize all layers in both VGG and GoogLeNet since it is conducive to the simplicity of forward computation. Moreover, quantization of small complexity layers would be helpful for seek the performance difference between two algorithms. Distribution of quantized weights is shown in figure 9 and 10. As we introduced in the former section, algorithm 1 penalizes all strong connections regardless of bitwidth.

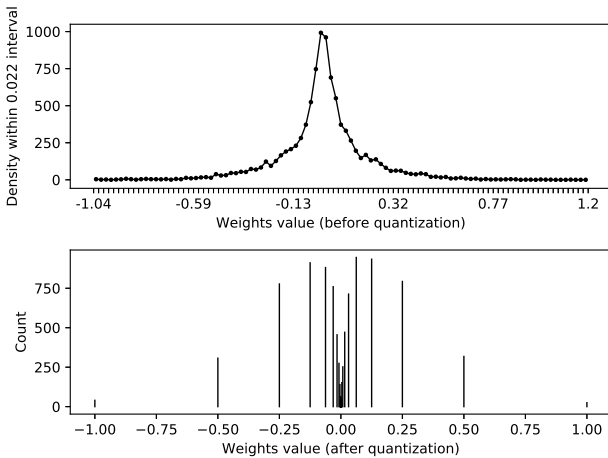


Figure 9: 4 bits quantization of the first layer in GoogLeNet via algorithm 1.

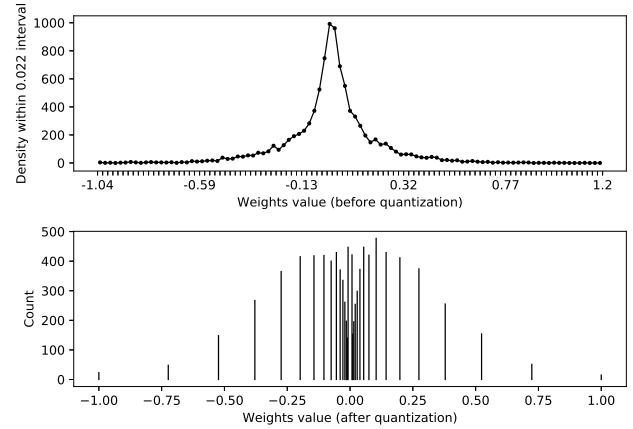


Figure 10: 4 bits quantization of the first layer in GoogLeNet via algorithm 2.

A tabular representation of results is shown in table 4. We set FSR parameter to 1 in algorithm 1. The result shows that our algorithm is in the lead in every item. Particularly, 6 bits quantization via our algorithm causes almost no loss on GoogLeNet. Since GoogLeNet consists of only 28 megabytes of weights, it is quite challenging to quantize it without retraining. To best of our knowledge, our algorithm achieved by far the minimum accuracy loss of quantized neural networks originates from GoogLeNet. We also expect that a better pre-trained GoogLeNet can increase the accuracy of the quantized version.

Table 4: Benchmark on GoogLeNet and VGG

Model	Alg1	Alg2	Ref
3 bits quantization			
GoogLeNet(Top-5)	0.7629	0.8021	0.8914
GoogLeNet(Top-1)	0.5143	0.560	0.6950
VGG	0.8894	0.8911	0.8993
4 bits quantization			
GoogLeNet(Top-5)	0.7593	0.8836	0.8914
GoogLeNet(Top-1)	0.5093	0.6607	0.6950
VGG	0.8942	0.8957	0.8993
6 bits quantization			
GoogLeNet(Top-5)	0.7593	0.8886	0.8914
GoogLeNet(Top-1)	0.5093	0.6914	0.6950
VGG	0.8910	0.8962	0.8993

5 CONCLUSION

In this paper, we have analysed the disadvantages of existing logarithmic quantization algorithm and proposed a new algorithm that simple yet powerful. We applied our algorithm to all layers of several neural networks showing its generality of handling layers of both large and small complexity.

We also pointed out that quantization error is not suitable for evaluating performance of quantization algorithms since it treats

strong and weak connections equal. Seeking a better standard of measurement would be set as our future work.

REFERENCES

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <http://tensorflow.org/> Software available from tensorflow.org.
- [2] Jingyong Cai, Masashi Takemoto, and Hironori Nakajo. 2018. Implementation of DNN on a RISC-V Open Source Microprocessor for IoT Devices. In *2018 IEEE 7th Global Conference on Consumer Electronics (GCCE) (IEEE GCCE 2018)*. Nara, Japan.
- [3] Gal Chechik, Isaac Meilijson, and Eytan Ruppin. 1998. Synaptic Pruning in Development: A Computational Account. *Neural Computation* 10, 7 (1998), 1759–1777. <https://doi.org/10.1162/089976698300017124> arXiv:<https://doi.org/10.1162/089976698300017124>
- [4] Jia Deng, Wei Dong, Richard Socher, Li jia Li, Kai Li, and Li Fei-fei. 2009. Imagenet: A large-scale hierarchical image database. In *In CVPR*.
- [5] Li Deng. 2012. The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]. 29 (11 2012), 141–142.
- [6] Google. [n. d.]. TensorFlow Lite. <https://www.tensorflow.org/lite/>
- [7] Song Han, Huizi Mao, and William J. Dally. 2015. Deep Compression: Compressing Deep Neural Network with Pruning, Trained Quantization and Huffman Coding. *CoRR abs/1510.00149* (2015). arXiv:1510.00149 <http://arxiv.org/abs/1510.00149>
- [8] Song Han, Jeff Pool, John Tran, and William J. Dally. 2015. Learning both Weights and Connections for Efficient Neural Networks. In *NIPS*.
- [9] Seyyed Hossein Hasanpour, Mohammad Rouhani, Mohsen Fayyaz, and Mohammad Sabokrou. 2016. Lets keep it simple, Using simple architectures to outperform deeper and more complex architectures. *arXiv preprint arXiv:1608.06037* (2016).
- [10] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized Neural Networks. In *Advances in Neural Information Processing Systems 29*, D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett (Eds.). Curran Associates, Inc., 4107–4115. <http://papers.nips.cc/paper/6573-binarized-neural-networks.pdf>
- [11] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations. *CoRR abs/1609.07061* (2016). arXiv:1609.07061 <http://arxiv.org/abs/1609.07061>
- [12] Forrest N. Iandola, Matthew W. Moskewicz, Khalid Ashraf, Song Han, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <1MB model size. *CoRR abs/1602.07360* (2016). arXiv:1602.07360 <http://arxiv.org/abs/1602.07360>
- [13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew G. Howard, Hartwig Adam, and Dmitry Kalenichenko. 2017. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *CoRR abs/1712.05877* (2017). arXiv:1712.05877 <http://arxiv.org/abs/1712.05877>
- [14] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. Caffe: Convolutional Architecture for Fast Feature Embedding. *arXiv preprint arXiv:1408.5093* (2014).
- [15] Eric Jones, Travis Oliphant, Pearu Peterson, et al. 2001–. SciPy: Open source scientific tools for Python. <http://www.scipy.org/> [Online; accessed <today>].
- [16] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip HS Torr. 2018. SNIP: Single-shot Network Pruning based on Connection Sensitivity. *arXiv preprint arXiv:1810.02340* (2018).
- [17] Hao Li, Soham De, Zheng Xu, Christoph Studer, Hanan Samet, and Tom Goldstein. 2017. Training Quantized Nets: A Deeper Understanding. In *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.). Curran Associates, Inc., 5811–5821. <http://papers.nips.cc/paper/7163-training-quantized-nets-a-deeper-understanding.pdf>
- [18] Timothy P Lillcrap, Daniel Cownden, Douglas B Tweed, and Colin J Akerman. 2016. Random synaptic feedback weights support error backpropagation for deep learning. *Nature communications* 7 (2016), 13276.
- [19] Jian-Hao Luo, Jianxin Wu, and Weiyao Lin. 2017. ThiNet: A Filter Level Pruning Method for Deep Neural Network Compression. *CoRR abs/1707.06342* (2017). arXiv:1707.06342 <http://arxiv.org/abs/1707.06342>
- [20] Daisuke Miyashita, Edward H. Lee, and Boris Murmann. 2016. Convolutional Neural Networks using Logarithmic Data Representation. *CoRR abs/1603.01025* (2016). arXiv:1603.01025 <http://arxiv.org/abs/1603.01025>
- [21] Hesham Mostafa, Vishwajith Ramesh, and Gert Cauwenberghs. 2017. Deep supervised learning using local errors. *CoRR abs/1711.06756* (2017). arXiv:1711.06756 <http://arxiv.org/abs/1711.06756>
- [22] Arild Nøkland. 2016. Direct feedback alignment provides learning in deep neural networks. In *Advances in neural information processing systems*. 1037–1045.
- [23] Karen Simonyan and Andrew Zisserman. 2014. Very Deep Convolutional Networks for Large-Scale Image Recognition. *CoRR abs/1409.1556* (2014). arXiv:1409.1556 <http://arxiv.org/abs/1409.1556>
- [24] Wonyong Sung, Sungho Shin, and Kyuhyeon Hwang. 2015. Resiliency of Deep Neural Networks under Quantization. *CoRR abs/1511.06488* (2015). arXiv:1511.06488 <http://arxiv.org/abs/1511.06488>
- [25] Seiya Tokui, Kenta Oono, Shohei Hido, and Justin Clayton. 2015. Chainer: a Next-Generation Open Source Framework for Deep Learning. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Twenty-ninth Annual Conference on Neural Information Processing Systems (NIPS)*. http://learningsys.org/papers/LearningSys_2015_paper_33.pdf
- [26] Frederick Tung, Greg Mori, and Simon Fraser. 2018. CLIP-Q : Deep Network Compression Learning by In-Parallel Pruning-Quantization.
- [27] Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. 2017. Incremental Network Quantization: Towards Lossless CNNs with Low-Precision Weights. *CoRR abs/1702.03044* (2017). arXiv:1702.03044 <http://arxiv.org/abs/1702.03044>
- [28] Aojun Zhou, Anbang Yao, Kuan Wang, and Yurong Chen. 2018. Explicit Loss-Error-Aware Quantization for Low-Bit Deep Neural Networks. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [29] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. 2016. DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients. *CoRR abs/1606.06160* (2016). <http://arxiv.org/abs/1606.06160>

6 APPENDIX

Listing 2: Quantization kernel

```
import numpy
class LogQuant:
    def __init__(self, layer, bitwidth=4):
        self.layer_data = layer
        self.width = bitwidth
        self.sign = numpy.sign(layer)
        self.lookup = numpy.linspace(0, -7, 2**self.width)
    def __round(self, x):
        idx = (numpy.abs(self.lookup - x)).argmin()
        return idx
    @property
    def log_quantized(self):
        round = numpy.vectorize(self.__round)
        return numpy.array(round(numpy.log2(numpy.abs(self.layer_data))), dtype=numpy.int8)
    @property
    def de_quantized(self):
        x = numpy.power(2.0, self.lookup[self.log_quantized])
        x = numpy.array(x, dtype=numpy.float32)
        return x * self.sign
```
